

Iris Dataset Project Summary

1. Data Preparation

- Loaded the Iris dataset using Seaborn's built-in function `sns.load_dataset("iris")`.
- Verified the dataset shape and basic structure using `.head()` and `.describe()`.
- Checked for missing values using `.isnull().sum()` and found none.
- Identified and removed duplicate rows with `.duplicated().sum()` and `drop_duplicates()` to ensure clean data.

2. EDA (Exploratory Data Analysis)

- Created histogram plots for each feature (`sepal_length`, `sepal_width`, `petal_length`, `petal_width`) to understand their distributions across different species (`setosa`, `versicolor`, `virginica`).
- Used a correlation heatmap to examine how features are interrelated:
 - Found strong correlation between `petal_length` and `petal_width`.
 - `sepal_width` showed weaker correlation with other features.
- These insights guided the importance of each feature in classification tasks.

3. Preprocessing

- Converted the categorical target (`species`) into numerical labels using `LabelEncoder` to enable model training.
- Scaled feature values to have zero mean and unit variance using `StandardScaler` to ensure all features contribute equally.
- For neural network modeling, converted the labels into one-hot encoded format using `to_categorical()`.

4. Modeling

4.1 Support Vector Machines (SVM)

- Trained SVM classifiers using three different kernels:
 - `linear`: for simple decision boundaries.
 - `poly`: for polynomial decision boundaries.
 - `rbf`: for capturing complex, non-linear relationships.
- Evaluated model performance on test data using confusion matrices and classification reports.
- Stored performance metrics (accuracy, precision, recall, F1-score) for each kernel.

4.2 Neural Networks

- Built a feedforward neural network using Keras with:
 - 2 hidden layers (16 and 8 neurons).

- Different activation functions tested: `relu`, `sigmoid`, `tanh`.
- Output layer used `softmax` for multi-class classification.
- Compiled with `adam` optimizer and `categorical_crossentropy` loss.
- Trained for 50 epochs and monitored validation loss to assess generalization.

5. Evaluation

- Compared SVM and Neural Network models using:
 - Accuracy (overall correctness)
 - Precision (exactness)
 - Recall (completeness)
 - F1-score (harmonic mean of precision and recall)
- Plotted confusion matrices to visualize misclassifications.
- Visualized training/validation loss curves for neural networks.
- Generated and compared **ROC curves** for each class.

6. Conclusion

- Both models performed well on the Iris dataset.
- **Best SVM kernel:** RBF — effectively handled the non-linear class boundaries.
- **Best Neural Network:** ReLU-based model — provided the highest accuracy and best convergence.
- No overfitting.